

Claude Skill Creation Study Guide

Created: 2026-02-01

Goal: Master creating custom tools (skills) for Claude via Anthropic's Tool Use API

Time: 45-60 minutes

■ What You'll Learn

1. What Claude tools/skills are and why they matter
 2. Tool schema design (JSON schema basics)
 3. How Claude executes tools (request/response cycle)
 4. Best practices for tool design
 5. Real-world examples (calculator, database, API calls)
 6. Advanced patterns (chaining, error handling, streaming)
 7. Testing and debugging tools
-

■ Part 1: Core Concepts (10 min)

What Are Claude Skills/Tools?

Skills = Custom functions/capabilities you give Claude that extend beyond text generation.

Without tools: - Claude can only generate text - No access to external data - No ability to perform actions - Limited to knowledge cutoff

With tools: - Query databases - Call external APIs - Perform calculations - Read/write files - Execute code - Control systems

Key insight: Tools transform Claude from "text generator" to "agent that can DO things."

How Tools Work

The Tool Use Cycle:

1. You: Send message + tool definitions to Claude
↓
2. Claude: Analyzes request, decides which tool(s) to use
↓
3. Claude: Returns `tool_use` block with tool name + parameters
↓
4. You: Execute the tool with provided parameters
↓
5. You: Send tool result back to Claude
↓
6. Claude: Processes result, continues reasoning
↓
7. Claude: Either uses more tools OR returns final answer

Important: YOU execute the tools, not Claude. Claude just tells you what to call.

■ Part 2: Tool Schema Basics (15 min)

Tool Definition Structure

Every tool needs 3 things:

```
{  
  "name": "tool_name",  
  "description": "What this tool does (Claude reads this!)",  
  "input_schema": {  
    "type": "object",  
    "properties": {  
      "param1": {  
        "type": "string",  
        "description": "What this parameter is for"  
      }  
    }  
  }  
}
```

```

    },
    "param2": {
      "type": "number",
      "description": "Another parameter"
    }
  },
  "required": ["param1"]
}

```

3 Key Parts:

1. **name** - Identifier (snake_case, descriptive)
2. **description** - HOW Claude decides to use this tool (critical!)
3. **input_schema** - JSON Schema defining parameters

Example 1: Simple Calculator

```

{
  "name": "calculator",
  "description": "Performs basic arithmetic operations. Use this when the user asks for calcula",
  "input_schema": {
    "type": "object",
    "properties": {
      "operation": {
        "type": "string",
        "enum": ["add", "subtract", "multiply", "divide"],
        "description": "The arithmetic operation to perform"
      },
      "a": {
        "type": "number",
        "description": "First number"
      },
      "b": {
        "type": "number",
        "description": "Second number"
      }
    },
    "required": ["operation", "a", "b"]
  }
}

```

```
}
```

Usage: - User: "What's 15 * 23?" - Claude decides: Use calculator tool - Claude returns: `{"operation": "multiply", "a": 15, "b": 23}` - You execute: `15 * 23 = 345` - You send back: `{"result": 345}` - Claude responds: "15 multiplied by 23 equals 345."

Example 2: Weather API

```
{
  "name": "get_weather",
  "description": "Fetches current weather for a given location. Use this when user asks about w",
  "input_schema": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "City name or coordinates (e.g., 'London' or '51.5074,-0.1278')"
      },
      "units": {
        "type": "string",
        "enum": ["celsius", "fahrenheit"],
        "description": "Temperature units",
        "default": "celsius"
      }
    },
    "required": ["location"]
  }
}
```

Implementation (Python):

```
def execute_get_weather(location, units="celsius"):
    # Call weather API
    response = requests.get(f"https://api.weather.com/v1/current?location={location}&units={units}")
    data = response.json()
    return {
        "temperature": data["temp"],
        "condition": data["condition"],
        "humidity": data["humidity"]
    }
```

JSON Schema Types

Type	Description	Example
<code>string</code>	Text value	<code>"London"</code>
<code>number</code>	Integer or float	<code>42, 3.14</code>
<code>integer</code>	Whole number only	<code>42</code>
<code>boolean</code>	True/false	<code>true</code>
<code>array</code>	List of values	<code>["apple", "banana"]</code>
<code>object</code>	Nested structure	<code>{"name": "John", "age": 30}</code>
<code>enum</code>	Limited choices	<code>["red", "green", "blue"]</code>

Validation Keywords: - `required`: Array of mandatory fields - `enum`: Limited set of values - `minimum/maximum`: Number constraints - `minLength/maxLength`: String constraints - `pattern`: Regex pattern for strings - `default`: Default value if not provided

■ Part 3: The Request/Response Cycle (10 min)

Step 1: Sending a Request with Tools

Python SDK:

```

import anthropic

client = anthropic.Anthropic(api_key="your-api-key")

message = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    tools=[
        {
            "name": "get_weather",
            "description": "Gets weather for a location",
            "input_schema": {
                "type": "object",
                "properties": {
                    "location": {"type": "string", "description": "City name"}
                },
                "required": ["location"]
            }
        }
    ],
    messages=[
        {"role": "user", "content": "What's the weather in Paris?"}
    ]
)

print(message)

```

Step 2: Claude's Response (Tool Use)

Response structure:

```

{
  "id": "msg_123",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "I'll check the weather for you."
    },
    {
      "type": "tool_use",

```

```

        "id": "toolu_456",
        "name": "get_weather",
        "input": {
            "location": "Paris"
        }
    },
    "stop_reason": "tool_use"
}

```

Key fields: - `content[].type`: "text" or "tool_use" - `tool_use.name`: Which tool to execute - `tool_use.input`: Parameters to pass - `tool_use.id`: Unique ID for this tool call (you'll need this!) - `stop_reason`: "tool_use" means Claude wants you to execute tools

Step 3: Execute the Tool

Your code:

```

# Extract tool use from response
tool_use = None
for block in message.content:
    if block.type == "tool_use":
        tool_use = block
        break

# Execute the tool
if tool_use.name == "get_weather":
    result = execute_get_weather(**tool_use.input)
    # result = {"temperature": 18, "condition": "Cloudy", "humidity": 65}

```

Step 4: Send Tool Result Back

Continue conversation:

```

response = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    tools=[...], # Same tools as before
    messages=[

```

```

    {"role": "user", "content": "What's the weather in Paris?"},
    {"role": "assistant", "content": message.content}, # Claude's tool_use
    {
        "role": "user",
        "content": [
            {
                "type": "tool_result",
                "tool_use_id": tool_use.id, # Match the ID!
                "content": json.dumps(result)
            }
        ]
    }
]
)

```

Step 5: Claude's Final Response

Response:

```

{
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "The weather in Paris is currently cloudy with a temperature of 18°C and 65% humidity."
    }
  ],
  "stop_reason": "end_turn"
}

```

stop_reason: "end_turn" = Claude is done, no more tools needed.

■ Part 4: Best Practices (10 min)

1. Write Clear Descriptions

■ **Bad:**


```
{
  "name": "get_data",
  "description": "Gets data"
}
```

■ Good:

```
{
  "name": "get_user_profile",
  "description": "Retrieves user profile data from the database by user ID. Use this when you need user information."
}
```

Why: Claude uses descriptions to decide WHEN to use a tool. Be specific!

2. Use Descriptive Parameter Names

■ Bad:

```
"properties": {
  "x": {"type": "string"},
  "y": {"type": "number"}
}
```

■ Good:

```
"properties": {
  "user_id": {
    "type": "string",
    "description": "Unique identifier for the user"
  },
  "include_metadata": {
    "type": "boolean",
    "description": "Whether to include additional metadata in the response",
    "default": false
  }
}
```

3. Validate Input in Your Implementation

Always validate:

```
def execute_tool(params):  
    # Validate  
    if not params.get("user_id"):  
        return {"error": "user_id is required"}  
  
    if not isinstance(params["user_id"], str):  
        return {"error": "user_id must be a string"}  
  
    # Execute  
    user = db.get_user(params["user_id"])  
    if not user:  
        return {"error": "User not found"}  
  
    return {"success": True, "data": user}
```

Return errors clearly - Claude will see them and can recover.

4. Return Structured Results

■ Bad:

```
return "Temperature is 18 degrees and it's cloudy"
```

■ Good:

```
return {  
    "temperature": 18,  
    "units": "celsius",  
    "condition": "cloudy",  
    "humidity": 65,  
    "timestamp": "2026-02-01T13:00:00Z"  
}
```

Why: Structured data lets Claude reason better and extract specific values.

5. Handle Errors Gracefully

Return error info Claude can understand:

```

try:
    result = api_call()
    return {"success": True, "data": result}
except APIError as e:
    return {
        "success": False,
        "error": str(e),
        "code": e.status_code,
        "retryable": e.status_code in [429, 500, 503]
    }

```

Claude can then decide to retry or inform the user.

6. Use Enums for Limited Choices

Good for:

```

{
  "operation": {
    "type": "string",
    "enum": ["add", "subtract", "multiply", "divide"]
  },
  "sort_order": {
    "type": "string",
    "enum": ["asc", "desc"]
  }
}

```

Prevents invalid inputs - Claude won't try to pass "addition" when you expect "add".

7. Provide Defaults for Optional Parameters

```

{
  "limit": {
    "type": "integer",
    "description": "Maximum number of results to return",
    "default": 10,
    "minimum": 1,
    "maximum": 100
  }
}

```

```
}  
}
```

8. Document Complex Tools Well

Example - Database query tool:

```
{  
  "name": "query_database",  
  "description": "Executes a SQL query on the analytics database. Use this to retrieve data about users, products, and orders.",  
  "input_schema": {  
    "type": "object",  
    "properties": {  
      "query": {  
        "type": "string",  
        "description": "SQL SELECT query. Must be read-only. Example: 'SELECT * FROM users WHERE age > 30'",  
      },  
    },  
    "required": ["query"]  
  }  
}
```

Explicit constraints help Claude use tools correctly.

■ ■ Part 5: Real-World Examples (15 min)

Example 1: File System Tool

```
{  
  "name": "read_file",  
  "description": "Reads contents of a file from the filesystem. Use when user asks to read, view, or inspect a file.",  
  "input_schema": {  
    "type": "object",  
    "properties": {  
      "path": {  
        "type": "string",  
        "description": "File path to read, e.g., '/data/logs/app.log'"  
      },  
    },  
    "required": ["path"]  
  }  
}
```

```

        "description": "Absolute or relative path to the file"
    },
    "encoding": {
        "type": "string",
        "enum": ["utf-8", "ascii", "latin1"],
        "default": "utf-8"
    }
},
"required": ["path"]
}
}

```

Implementation:

```

def read_file(path, encoding="utf-8"):
    try:
        with open(path, 'r', encoding=encoding) as f:
            content = f.read()
        return {
            "success": True,
            "content": content,
            "size_bytes": len(content.encode(encoding))
        }
    except FileNotFoundError:
        return {"success": False, "error": f"File not found: {path}"}
    except PermissionError:
        return {"success": False, "error": f"Permission denied: {path}"}

```

Example 2: Database Query Tool

```

{
    "name": "query_users",
    "description": "Queries the users database. Use to find users by email, name, or registration",
    "input_schema": {
        "type": "object",
        "properties": {
            "email": {"type": "string", "description": "Filter by email address"},
            "name": {"type": "string", "description": "Filter by name (partial match)"},
            "registered_after": {"type": "string", "description": "ISO date (e.g., 2026-01-01)"},
            "limit": {"type": "integer", "default": 10, "minimum": 1, "maximum": 100}
        }
    }
}

```

```
}  
}  
}
```

Implementation:

```
def query_users(email=None, name=None, registered_after=None, limit=10):  
    query = "SELECT id, email, name, created_at FROM users WHERE 1=1"  
    params = []  
  
    if email:  
        query += " AND email = ?"  
        params.append(email)  
    if name:  
        query += " AND name LIKE ?"  
        params.append(f"%{name}%")  
    if registered_after:  
        query += " AND created_at > ?"  
        params.append(registered_after)  
  
    query += f" LIMIT {limit}"  
  
    results = db.execute(query, params).fetchall()  
    return {  
        "count": len(results),  
        "users": [dict(row) for row in results]  
    }
```

Example 3: Multi-Step Web Search

```
{  
    "name": "web_search",  
    "description": "Searches the web using Brave Search API. Use when user asks for current information",  
    "input_schema": {  
        "type": "object",  
        "properties": {  
            "query": {"type": "string", "description": "Search query"},  
            "count": {"type": "integer", "default": 5, "minimum": 1, "maximum": 10}  
        },  
        "required": ["query"]  
    }
```

```

    }
  },
  {
    "name": "fetch_webpage",
    "description": "Fetches and extracts readable content from a URL. Use after web_search to read",
    "input_schema": {
      "type": "object",
      "properties": {
        "url": {"type": "string", "description": "URL to fetch"}
      },
      "required": ["url"]
    }
  }
}

```

Tool chaining: 1. User: "What are the latest developments in quantum computing?" 2. Claude: Calls `web_search("latest quantum computing developments")` 3. You: Return search results with URLs 4. Claude: Analyzes results, calls `fetch_webpage("https://...")` 5. You: Return page content 6. Claude: Synthesizes final answer from multiple sources

Example 4: API Integration (Stripe)

```

{
  "name": "create_stripe_customer",
  "description": "Creates a new customer in Stripe. Use when setting up a new customer account",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "name": {"type": "string", "description": "Customer full name"},
      "metadata": {
        "type": "object",
        "description": "Additional key-value data to store"
      }
    },
    "required": ["email"]
  }
}

```

Implementation:

```
import stripe

def create_stripe_customer(email, name=None, metadata=None):
    try:
        customer = stripe.Customer.create(
            email=email,
            name=name,
            metadata=metadata or {}
        )
        return {
            "success": True,
            "customer_id": customer.id,
            "email": customer.email
        }
    except stripe.error.StripeError as e:
        return {
            "success": False,
            "error": str(e),
            "type": e.__class__.__name__
        }
```

■ Part 6: Advanced Patterns (10 min)

Pattern 1: Tool Chaining

Scenario: User asks "Find my last 5 orders and summarize them"

Tools: 1. `get_user_by_email(email)` → `user_id` 2. `get_orders(user_id, limit=5)` → order list 3. Claude synthesizes summary

Claude automatically chains:

```
Call 1: get_user_by_email("user@example.com")
← Result: {"user_id": "usr_123"}

Call 2: get_orders(user_id="usr_123", limit=5)
← Result: {"orders": [...]}
```



```
Final: "Your last 5 orders include..."
```

No extra code needed - Claude handles the flow!

Pattern 2: Conditional Tool Use

Scenario: Check if user exists, create if not

Tools: - `get_user(email)` - `create_user(email, name)`

Claude's reasoning:

```
User: "Set up account for john@example.com"
```

```
Claude thinks: First check if exists
```

```
→ Calls get_user("john@example.com")
```

```
← Result: {"error": "User not found"}
```

```
Claude thinks: User doesn't exist, create
```

```
→ Calls create_user("john@example.com", "John")
```

```
← Result: {"success": true, "user_id": "usr_456"}
```

```
Claude: "Account created for john@example.com"
```

Pattern 3: Parallel Tool Calls

Claude can call multiple tools simultaneously:

```
{
  "content": [
    {"type": "tool_use", "name": "get_weather", "input": {"location": "Paris"}},
    {"type": "tool_use", "name": "get_weather", "input": {"location": "London"}},
    {"type": "tool_use", "name": "get_weather", "input": {"location": "Berlin"}}
  ]
}
```

Your implementation:

```
# Execute all in parallel
import asyncio

async def execute_all_tools(tool_uses):
    tasks = [execute_tool_async(tu) for tu in tool_uses]
    return await asyncio.gather(*tasks)
```

Pattern 4: Streaming Tool Results

For long-running tools:

```
def execute_long_task(params):
    # Return initial status
    yield {"status": "starting", "progress": 0}

    # Process in chunks
    for i in range(10):
        process_chunk(i)
        yield {"status": "processing", "progress": (i+1)*10}

    # Final result
    yield {"status": "complete", "result": final_data}
```

Use case: Video processing, large file analysis, bulk operations

■ ■ Part 7: Common Pitfalls (5 min)

■ Mistake 1: Vague Descriptions

Problem:

```
{"description": "Gets stuff from database"}
```

Claude won't know when to use it!

Fix:

```
{"description": "Retrieves user account details from PostgreSQL by user ID. Use when you need a"}
```

■ Mistake 2: Missing Parameter Descriptions

Problem:

```
"properties": {  
  "id": {"type": "string"}  
}
```

Fix:

```
"properties": {  
  "user_id": {  
    "type": "string",  
    "description": "Unique user identifier (format: usr_XXXXX)"  
  }  
}
```

■ Mistake 3: Not Handling Errors

Problem:

```
def my_tool(param):  
    result = api.call(param) # Might fail!  
    return result
```

Fix:

```
def my_tool(param):  
    try:  
        result = api.call(param)  
        return {"success": True, "data": result}  
    except Exception as e:  
        return {"success": False, "error": str(e)}
```

■ Mistake 4: Returning Unstructured Text

Problem:

```
return "User John Doe, age 30, email john@example.com"
```

Fix:

```
return {  
  "name": "John Doe",  
  "age": 30,  
  "email": "john@example.com"  
}
```

■ Mistake 5: Forgetting tool_use_id

Problem:

```
# Sending tool result without matching ID  
{"type": "tool_result", "content": result} # Missing tool_use_id!
```

Fix:

```
{  
  "type": "tool_result",  
  "tool_use_id": original_tool_use.id, # Must match!  
  "content": json.dumps(result)  
}
```

■ Part 8: Testing & Debugging (5 min)

Test Your Tools Independently

Before integrating with Claude:

```
# Unit test your tool
def test_weather_tool():
    result = get_weather(location="London", units="celsius")
    assert result["temperature"] is not None
    assert result["condition"] in ["Sunny", "Cloudy", "Rainy"]
```

Log Tool Calls

```
def execute_tool(tool_name, params):
    print(f"[TOOL] {tool_name} called with {params}")
    result = TOOLS[tool_name](**params)
    print(f"[TOOL] {tool_name} returned {result}")
    return result
```

Test Claude's Tool Selection

Prompt variations: - "What's the weather in Paris?" → Should use `get_weather` - "Tell me about Paris" → Should NOT use `get_weather` - "Compare weather in Paris and London" → Should call `get_weather` twice

Debug Schema Issues

If Claude isn't using your tool: 1. Check description - is it clear WHEN to use? 2. Check parameter descriptions - are they helpful? 3. Try asking Claude directly: "Can you use the `get_weather` tool for Paris?"

If Claude passes wrong parameters: 1. Check parameter types match schema 2. Check enum values are clear 3. Add examples in description

■ Quick Reference

Minimal Tool Definition

```
{
  "name": "tool_name",
  "description": "What it does and when to use it",
  "input_schema": {
    "type": "object",
    "properties": {
      "param": {"type": "string", "description": "What this is"}
    },
    "required": ["param"]
  }
}
```

Complete Request Example (Python)

```
import anthropic
import json

client = anthropic.Anthropic(api_key="...")

def get_weather(location, units="celsius"):
    # Your implementation
    return {"temperature": 18, "condition": "Cloudy"}

# Initial request
response = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    tools=[{
        "name": "get_weather",
        "description": "Gets current weather for a location",
        "input_schema": {
            "type": "object",
            "properties": {
                "location": {"type": "string"}
            },
            "required": ["location"]
        }
    }],
    messages=[{"role": "user", "content": "What's the weather in Paris?"}]
)
```

```

# Extract tool use
tool_use = next((b for b in response.content if b.type == "tool_use"), None)

if tool_use:
    # Execute tool
    result = get_weather(**tool_use.input)

    # Send result back
    final = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=1024,
        tools=[...], # Same tools
        messages=[
            {"role": "user", "content": "What's the weather in Paris?"},
            {"role": "assistant", "content": response.content},
            {
                "role": "user",
                "content": [{
                    "type": "tool_result",
                    "tool_use_id": tool_use.id,
                    "content": json.dumps(result)
                }]
            }
        ]
    )

    print(final.content[0].text)

```

■ Your Action Plan

Quick Win (15 min)

Build a calculator tool: 1. Define the schema (above examples) 2. Implement the function 3. Test with Claude: "What's 123 * 456?"

Next Level (30 min)

Build a file system tool: 1. Define tools: `read_file`, `list_directory` 2. Implement safely (validate paths!) 3. Test: "What files are in my Documents folder?"

Advanced (1-2 hours)

Build a multi-tool research agent: 1. Tools: `web_search`, `fetch_url`, `save_note` 2. Prompt: "Research the latest in quantum computing and save a summary" 3. Watch Claude chain tools automatically

■ Resources

- **Official docs:** <https://docs.anthropic.com/en/docs/build-with-claude/tool-use>
- **JSON Schema:** <https://json-schema.org/>
- **Examples:** https://github.com/anthropics/anthropic-cookbook/tree/main/tool_use
- **Python SDK:** <https://github.com/anthropics/anthropic-sdk-python>

Study time: ~60 minutes

First implementation: ~15 minutes

ROI: Transform Claude from text generator → capable agent ■

Ready to build your first tool? Start with the calculator example!